ДИСЦИПЛИНА	Основы интеллектуального анализа данных
ИНСТИТУТ	Институт технологий управления
КАФЕДРА	Статистики и математических методов в управлении
ГРУППЫ	ГСБО-01-21, ГСБО-01-22,
ВИДЫ УЧЕБНОГО МАТЕРИАЛА	П6. Тема – Решение задачи классификации средствами языка R
ПРЕПОДАВАТЕЛЬ	Бурцева Т.А.
СЕМЕСТР	5 семестр

Методические указания

Выполнение работы предполагает реализацию примера на данных файла data5.xls, чтобы понять логику методов и получить практические навыки их применения в R-Studio, а также разработку собственных моделей. В качестве исходных данных для разработки собственных моделей предлагаются данные из файла data5.xls, предпочтительней использовать данные, которые студент выбрал сам. Выбор независимых переменных для самостоятельных моделей из файла data5.xls реализуется случайным образом (set=sample(1:10, round(0.2*10))), зависимая переменная Y одна для всех. Переменную Y нужно сделать бинарной, используя правило: если ее значение больше 1, то 1, если меньше, то 0. Классификация – это задача отнесения объекта к одному из заранее известных классов. Бывает бинарная и множественная классификация. В работе выполняется бинарная классификация, так как Y – бинарная переменная. Методами, которые используются в работе являются логистическая регрессия и нейросеть.

Логистическая регрессия – это разновидность множественной регрессии, общее назначение которой состоит в анализе связи между несколькими независимыми переменными (называемыми также регрессорами или предикторами) и зависимой переменной. Бинарная логистическая регрессия, как следует из названия, применяется в случае, когда зависимая переменная является бинарной (т.е. может принимать только два значения). Иными словами, с помощью логистической регрессии можно оценивать вероятность того, что событие наступит для конкретного испытуемого (больной/здоровый, возврат кредита/дефолт и т.д.).

$$y = a + b_1 x_1 + b_2 x_2 + \ldots + b_n x_n$$

Для решения проблемы задача регрессии может быть сформулирована иначе: вместо предсказания бинарной переменной, мы предсказываем непрерывную переменную со значениями на отрезке [0,1] при любых значениях независимых переменных. Это достигается применением следующего регрессионного уравнения (логит-преобразование):

$$p = \frac{1}{1+e^{-y}},$$

где P – вероятность того, что произойдет интересующее событие; e – основание натуральных логарифмов 2,71…; y – стандартное уравнение регрессии.

На рисунке 1 дана принципиальная модель нейрона. Математически нейрон представляет собой нелинейную функцию активации скалярного произведения вектора входных данных (input) и вектора весов (weight), при этом одно их входных значений называется смещением (bias) и фиксируется на значении 1. В качестве функции активации часто рассматривается логистическая функция, которая является примером сигмоиды.

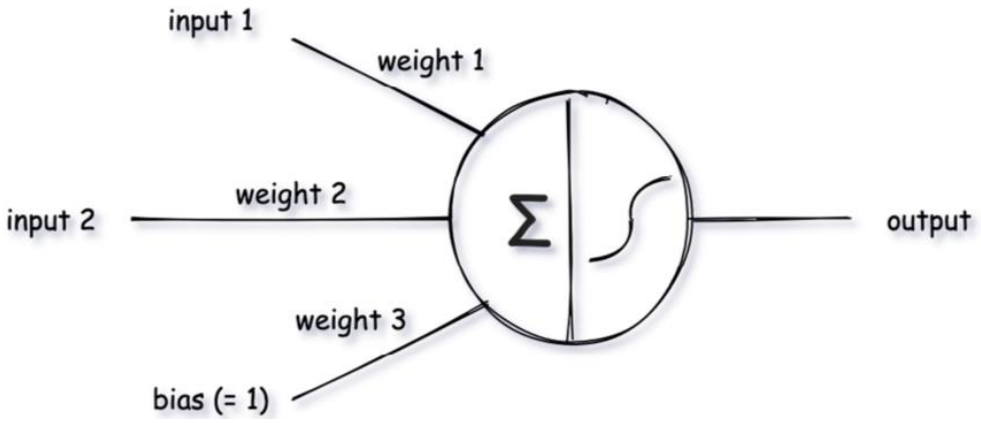


Рис. 1. Модель нейрона.

Математически нейрон представляет собой нелинейную функцию активации скалярного произведения вектора входных данных (input) и вектора весов (weight), при этом одно их входных значений называется смещением (bias) и фиксируется на значении 1. В качестве функции активации часто рассматривается логистическая функция, которая является примером сигмоиды. Математически нейрон искусственной нейронной сети описывается следующей функцией $\frac{1}{1+e^{-\vec{x}\vec{w}}}$. Таким образом, это не что иное, как логистическая регрессия, приведенная в вид обобщенной линейной модели (GLM), которая в свою очередь является логистической функцией линейной регрессии. Исходя из этого, в случае использования сигмоиды в качестве функции активации, можем провести равенство между тремя понятиями: нейрон, построенный на основе сигмоиды, логистическая регрессия и логистическая линейная регрессия, которая имеет много ценных свойств (монотонность и дифференцируемость, устойчивость к выбросам, простое выражение для ее производной и т.д.). Выходное значение сигмоидального нейрона лежит в диапазоне [0, 1]. При уменьшении коэффициента с сигмоид становится более пологим, вырождаясь в пределе при c=0 в горизонтальную линию на уровне 0.5. При увеличении с сигмоид приближается по внешнему виду к функции единичного скачка с порогом T в точке x=0. Для обучения нейронных сетей в среде R используются два пакета - neuralnet и nnet, обеспечивающие гибкие функциональные возможности построения моделей классификации и регрессии на основе многослойного персептрона.

Пример выполнения работы

Шаг 0. Загружаем нужные пакеты

```
#пакет построения графиков в R
install.packages("ggplot2")
#пакет для работы с нейросетью
install.packages("neuralnet")
install.packages("caret")
library(neuralnet)
library(ggplot2)
library(caret)
```

Шаг. 1. Загружаем данные из файла data5.xls, который содержит данные об индексах оценки региональной производительности труда и ее факторов.

```
#выбор номеров независимых переменных варианта
set=sample(1:10, round(0.2*10))
#создание таблицы данных
t=data5[,c(4,8,13)]
#создание бинарной переменной Y
```

```
for (i in 1:length(t$Y)){if (t$Y[i]>=1) {t$Y[i]=1}}
for (i in 1:length(t$Y)){if (t$Y[i]<1 ) {t$Y[i]=0}}
```
Шаг. 2 С помощью функции glm строим модель логистической регрессии
```
#построим модель логистической регрессии
fit  <- glm(t$Y ~ t$x2+t$x6+t$x2:t$x6, t, family = "binomial")
```
Результат:
```
Call:
glm(formula = t$Y ~ t$x2 + t$x6 + t$x2:t$x6, family = "binomial",
    data = t)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)   -23.58      32.43  -0.727    0.467
t$x2           23.85      32.47   0.735    0.463
t$x6           21.36      32.34   0.661    0.509
t$x2:t$x6     -21.67      32.34  -0.670    0.503

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 120.32  on 86  degrees of freedom
Residual deviance: 119.48  on 83  degrees of freedom
AIC: 127.48

Number of Fisher Scoring iterations: 4
```
Информационный критерий Акаике (AIC) $AIC = 2k + n \cdot \ln(RSS)$, где k – число параметров модели, n – число наблюдений в массиве данных, RSS – остаточная сумма квадратов. Считается, что наилучшей будет модель с наименьшим значением критерия AIC.
Шаг 3. Определим теоретические значения Yp.
```
#посчитаем logit(p)
t$prob<--23.58+23.85*t$x2+21.36*t$x6-21.67*t$x2*t$x6
#посчитаем модельное Yp
t$Yp<-exp(t$prob)/(1+exp(t$prob))
#сделаем Yp бинарным
for (i in 1:length(t$Yp)){if (t$Yp[i]>=0.5) {t$Yp[i]=1}}
for (i in 1:length(t$Yp)){if (t$Yp[i]<0.5 ) {t$Yp[i]=0}}
```
Шаг. 4. Посчитаем метрики качества бинарной классификации на основе логистической регрессии, а именно:
Точность (Accuracy) — отношение правильно классифицированных наблюдений ко всем. Accuracy = (TP + FP) / (P + N)
Полнота/чувствительность (Recall, True Positive Rate, Sensitivity) — отношение правильно определённых положительных случаев ко всем положительным (показывает какую часть положительных случаев модель правильно классифицирует). Recall = TP / (TP + FN)
Прецизионность (Precision) — отношение правильно определённых положительных наблюдений ко всем определённым как положительные. Precision = TP / (TP + FP)
F1-score — отношение удвоенного произведения Precision и Recall к их сумме.
F1 Score = (2 x Precision x Recall) / (Precision + Recall). Оценка F1 варьируется от 0 до 1, где значение 1 означает идеальную точность и отзыв, в то время как значение 0 указывает на низкую производительность.
Специфичность (Specificity, True Negative Rate) — отношение правильно определённых отрицательных случаев ко всем отрицательным (показывает какую часть отрицательных случаев модель правильно классифицирует). Specificity = TN / (TN + FP)
Оценка Каппа: Kappa = (observed - expected) / (100% - expected). Оценка Каппа, также известная как коэффициент каппа Коэна, представляет собой оценочную метрику, используемую в машинном обучении и статистике для измерения уровня соответствия между прогнозируемыми и фактическими метками классов сверх того, что можно было бы

ожидать случайно. Это особенно полезно при работе с несбалансированными наборами данных или когда распределение классов неравномерно. Оценка kappa варьируется от -1 до 1, где 1 указывает на идеальное соответствие, 0 указывает на случайное соответствие и отрицательные значения указывают на то, что согласие хуже, чем вероятность. Оценка каппа обычно используется в ситуациях, когда распределение классов несбалансировано или когда наблюдается значительный перекос в классе. Это обеспечивает более надежную меру согласия, принимая во внимание согласие, которого можно ожидать случайно.

Итак, аккуратность - доля правильных ответов модели, точность – доля правильных ответов модели в пределах класса, полнота – доля объектов класса к общему числу объектов.

```
# найдем Accuracy
j=0
for (i in 1:length(t$Yp)){if (t$Yp[i]==t$Y[i]) {j=j+1}}
Accuracy=j/length(t$Yp)
```

Результат:

```
Accuracy
[1] 0.5287356 – точно расклассифицированы 53% регионов
```

R считает матрицу , для оценки метрик качества, это функция confusionMatrix из пакета caret.

	TP	FP
FN	0	0
TN	0	0

```
cm<-confusionMatrix(t$Y, t$Yp)
```

Результат: `cm`

```
     [,1] [,2]
[1,]    0    0
[2,]    0   46
```

В примере можно оценить только Accuracy.

Шаг 5. Решим задачу бинарной классификации на основе нейросети, используя x9, x10 и Y.

```
#создадим объект данных iris
iris=data5[,11:13]
#разобьём их на 2 части, обучающую и тестовую выборку, чтобы реализовать кросс-прове
рку на тестовой выборке.
set.seed(101)
ind <- sample(2, nrow(iris), replace = TRUE, prob = c(0.7, 0.3))
trainset <- iris[ind == 1, ]
testset <- iris[ind == 2, ]
#сделаем дополнительные логические переменные YT (ответ на условие Y<=1), YF наобо
рот.
trainset$YT=trainset$Y<=1
trainset$YF=trainset$Y>1
#округлим значения переменных x до десятого знака
trainset$x9=round(trainset$x9, 1)
trainset$x10=round(trainset$x10, 1)
#построим нейросеть с одним скрытым слоем, содержащим два нейрона
net.iris <- neuralnet( (YT+YF)~x9+x10 , trainset, hidden = 2 ) # hidden = 2, так как 2 градации
#y Y
 plot(net.iris)
```

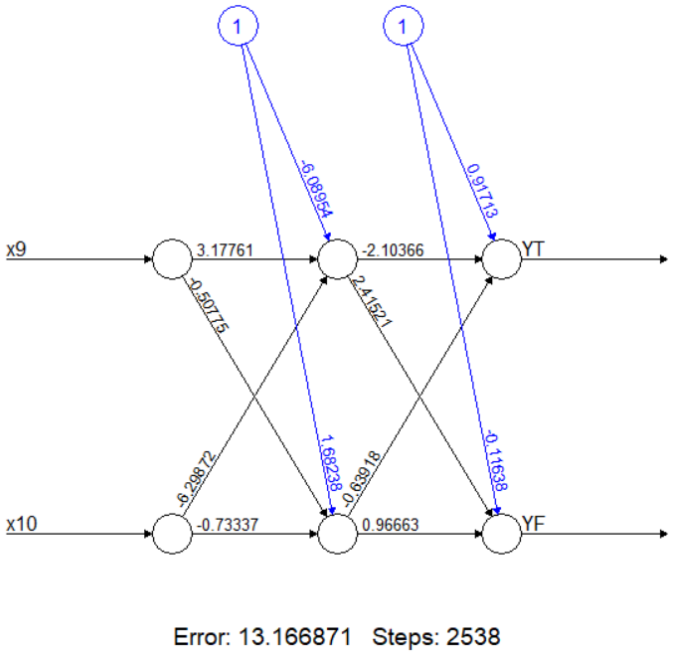


Рис. 2. График обученной нейронной сети

На рис. 2 видно, что на выходной слой поступает 2 сигнала, которые представляют собой скомбинированные и трансформированные значения двух исходных переменных. С их использованием и на основе обобщенных весов и функций активации последнего слоя оцениваются вероятности отнесения регионов к каждому из двух видов (рост или спад производительности труда).

```
#посмотрим результаты
net.iris$result.matrix
                              [,1]
error                 1.316687e+01
reached.threshold     9.908858e-03
steps                 2.538000e+03
Intercept.to.1layhid1 -6.089541e+00
x9.to.1layhid1         3.177611e+00
x10.to.1layhid1       -6.298716e+00
Intercept.to.1layhid2  1.682383e+00
x9.to.1layhid2        -5.077493e-01
x10.to.1layhid2       -7.333664e-01
Intercept.to.YT        9.171345e-01
1layhid1.to.YT        -2.103664e+00
1layhid2.to.YT        -6.391830e-01
Intercept.to.YF       -1.163783e-01
1layhid1.to.YF         2.415213e+00
1layhid2.to.YF         9.666280e-01
```

Из сводки итоговых результатов видно, что процесс обучения нуждался в нескольки х тысячах шагов (steps), пока максимумы всех частных производных функции ошибок reac hed.threshold по абсолютной величине не оказались ниже чем 0.01 (порог по умолчанию). Ошибка error оценивается как уровень правдоподобия по критерию AIC. Остальные велич ины, представленные в net.iris$result.matrix и net.iris$generalized.weights, содержат весовые коэффициенты w и значения сигмоид c для каждого нейрона, которые можно должным образом проанализировать и интерпретировать.

```
# считаем по тестовой выборке, используя нейросеть
#округляем
testset$x9=round(testset$x9, 1)
testset$x10=round(testset$x10, 1)
#считаем
```

```
net.prob <- predict(net.iris, testset[,1:2])
#сделаем У бинарным в testset
for (i in 1:length(testset$Y)){if (testset$Y[i]>=0.5) {testset$Y[i]=1}}
for (i in 1:length(testset$Y)){if (testset$Y[i]<0.5) {testset$Y[i]=0}}
pred = c(TRUE,   FALSE)[apply(net.prob, 1, which.max)]
(table(Факт = testset$Y, Прогноз = pred))
```

Результат:

```
Прогноз
Факт FALSE TRUE
   1     6   27
```

Считаем точность классификации

```
Acc = mean(pred == testset$Y)
paste("Точность=", round(100*Acc, 2), "%", sep = "")
```

Результат: `[1] "Точность=81.82%"`

Вывод: точность нейросети выше.